Memory-Aware CUDA Acceleration of Graph-Based Financial Fraud Detection Using Parallel Neighbor Aggregation

Praveen Kanth N, BharaniKumar J, Kishore B, and Kovendan A. K. P, *Senior Member, IEEE*

Manuscript received October 31, 2026; revised November 15, 2026; accepted December 1, 2026. Date of publication December 8, 2026. This research was conducted within the Department of Database Systems, School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore, India. (Corresponding author: Kovendan A. K. P.) Praveen Kanth N (Reg: 23MIC0078), BharaniKumar J (Reg: 23MIC0039), and Kishore B (Reg: 23MIC0050) are with SCOPE, Vellore Institute of Technology (VIT), Vellore 632014, India (e-mail: praveen.kanth2023@vitstudent.ac.in; bharanikumar.j2023@vitstudent.ac.in; kishore.b2023@vitstudent.ac.in). Dr. Kovendan A. K. P. is Assistant Professor Sr. Grade 2 with the Department of Database Systems, School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore 632014, India (e-mail: kovendan.akp@vit.ac.in).

Abstract*—Real-time financial fraud detection is constrained by an acute architectural tension: while relational graph representations capture collusive synthetic identities that evade tabular surveillance, topological neighbor aggregation induces severe memory-bandwidth saturation on modern hardware. Because graph neighbor gathering exhibits an arithmetic intensity well below 0.10 FLOP/byte, central processing units spend up to 95% of execution cycles waiting on dynamic random-access memory pointer traversals. In this article, we propose FraudGraph Intelligence (FGI), a hardware-conscious compute framework engineered to accelerate topological feature propagation on massive transaction networks. We formulate a compact Compressed Sparse Row (CSR) invariant that compresses 10,000 transaction nodes and 231,984 directed causal edges into contiguous memory structures under 1.5 MB. We architect four progressive compute tiers on NVIDIA graphics hardware: a baseline thread-parallel model, a warp-coalesced aggregator that maps contiguous 32-dimensional feature contraction into unified 128-byte DRAM bus requests, a degree-aware dynamic dispatch mechanism with shared-memory reduction trees for power-law hubs (dmax = 252), and a double-buffered asynchronous direct memory access (DMA) streaming pipeline that overlaps PCIe host-to-device transfers with compute. Benchmarked on an NVIDIA Tesla T4 GPU, FGI achieves an end-to-end kernel aggregation latency of 0.0581 ms and yields a 44.60× speedup over reference scalar execution. Downstream machine learning evaluation under strict causal streaming constraints reveals that maximum (MAX) neighbor enrichment lifts Logistic Regression PR-AUC from 0.3088 to 0.3315 (+7.34% relative gain) and expands fraud recall from 37.7% to 45.3% on severe 3.5% class imbalance. All kernels, schemas, and empirical logs are released reproducibly.

***Index Terms*—**CUDA Acceleration, Financial Fraud Detection, Graph Neural Networks, Parallel Computing, Memory Bandwidth Bottlenecks, Compressed Sparse Row, Warp Coalescing.

I. INTRODUCTION

FINANCIAL fraud surveillance systems process hundreds of thousands of electronic credit card, debit, and point-of-sale transactions per second. Historically, commercial anti-money laundering (AML) platforms and risk engines have evaluated transactions as isolated statistical instances [1]. Under this tabular paradigm, machine learning models scrutinize scalar attributes such as monetary expenditure, temporal delta from previous purchase, merchant category code, and geolocation deviation. While computationally economical, tabular classification exhibits a fundamental blind spot: it cannot detect decentralized fraud rings, credit-mule syndicates, and coordinated identity recycling schemes [2], [3]. Organized adversaries deliberately structure fraudulent withdrawals into small, sub-threshold bursts distributed across multiple accounts, thereby mimicking benign consumer behavior when viewed in isolation.

To bridge this surveillance gap, graph-based learning frameworks model financial interactions as relational networks [4], [5]. Nodes designate transactions, payment cards, bank accounts, or physical terminal devices, whereas directed edges denote temporal transfers, common card numbers, or identical hardware fingerprints. Within this topological formulation, synthetic fraud rings manifest as dense cliques, cyclic money routings, or anomalous bipartite subgraphs. Graph representation methods—specifically the neighbor aggregation stages of Graph Convolutional Networks (GCNs) [6], GraphSAGE [7], and Graph Attention Networks (GATs) [8], [9]—enrich a target transaction node by iteratively aggregating latent feature vectors from its structural neighborhood.

Despite their algorithmic efficacy, graph-based methods encounter severe architectural bottlenecks when deployed in production payment rails [10]–[12]. Graph neighbor gathering is fundamentally memory-bandwidth bound rather than compute bound [13]. Unlike dense matrix arithmetic or deep convolutional layers, which achieve arithmetic intensities exceeding 100 FLOPs per byte transferred, sparse neighbor aggregation exhibits an arithmetic intensity frequently below 0.05 FLOP/byte. On standard multi-core central processing units (CPUs), neighbor gathering degenerates into irregular pointer chasing across disparate regions of dynamic random-access memory (DRAM). Because graph connectivity is non-local, CPU data caches experience miss rates exceeding 85%, leaving processing pipelines idle while awaiting memory line fills.

Furthermore, financial transaction graphs exhibit power-law degree distributions governed by scale-free phenomena [14]. While typical consumer accounts generate fewer than ten transactions monthly, merchant payment gateways and collusive mule accounts act as massive topological hubs with degrees spanning hundreds or thousands of connections. When mapped naively to Single Instruction, Multiple Threads (SIMT) architectures such as NVIDIA Graphics Processing Units (GPUs), these structural hubs induce catastrophic warp divergence and memory serialization [15], [16]. Threads assigned to low-degree vertices complete instantaneously and sit idle, while peer threads within the identical 32-thread warp serialize across hundreds of sequential memory loads, destroying Streaming Multiprocessor (SM) efficiency.

To resolve these micro-architectural bottlenecks, this article introduces FraudGraph Intelligence (FGI), an end-to-end, memory-aware CUDA acceleration framework for financial transaction graph feature enrichment. The core contributions of this study are structured as follows:

1) Causal Compressed Sparse Row (CSR) Layout: We design a zero-copy contiguous CSR memory layout storing 10,000 transactions and 231,984 directed edges within 1.4 MB of unified storage,

eliminating dynamic memory allocations and establishing strict causal non-anticipative invariants (tu < tv).

2) Tiered CUDA Acceleration Kernels: We engineer four micro-architectural GPU execution tiers: a thread-parallel baseline, a warp-coalesced aggregator that maps contiguous 32-dimensional feature contraction into unified 128-byte memory transactions, a degree-aware dynamic dispatch mechanism utilizing shared-memory parallel reduction trees, and an asynchronous dual-stream pipeline utilizing page-locked host memory to overlap PCIe bus transfers.

3) Extensive Micro-Architectural Profiling: We conduct rigorous hardware benchmarking on NVIDIA Tesla T4 and AMD64 architectures, establishing a 44.60× speedup for vectorized Sparse Matrix-Matrix Multiplication (SpMM) on CPU and an end-to-end latency of 0.0581 ms on GPU, alongside a verified empirical root-cause analysis of floating-point IEEE 754 non-associativity in degree-aware summation.

4) Causal Downstream ML Evaluation: We evaluate Logistic Regression and Random Forest classifiers across five topological representations under discrete validation thresholding (ADR-007), proving that MAX neighbor enrichment delivers a 7.34% relative boost in PR-AUC (0.3088 → 0.3315) and elevates fraud recall from 37.7% to 45.3% under extreme 3.5% class imbalance.

II. MATHEMATICAL FORMULATION AND CSR TOPOLOGY

A. Causal Transaction Graph Definition

Let G = (V, E, X, y, t) represent a directed, attributed transaction graph observed over a temporal window Tobs. The vertex set V = {v0, v1, ..., vN-1} represents N individual financial transaction instances sorted in monotonic chronological sequence according to transaction timestamps t(vi) in R+. Each vertex vi in V is parameterized by an input feature vector xi in R^D and a binary classification target yi in {0, 1}, where yi = 1 denotes synthetic fraudulent activity and yi = 0 denotes legitimate banking activity.

The directed edge set E subseteq V x V encodes relational dependencies between transactions. A directed edge e = (u, v) is instantiated if and only if transaction u shares an underlying entity identifier with transaction v (e.g., matching primary account number or shared device hash) and satisfies the strict non-anticipative causality condition:

$$(u, v) \in E \iff (u \sim_{entity} v) \wedge (0 < tv - tu \leq \Delta t_{window}), \quad (1)$$

where Δtwindow represents the historical surveillance lookback horizon (set to 7.0 days, or 604,800 seconds). The strict temporal inequality tu < tv guarantees that the resulting directed graph is a Directed Acyclic Graph (DAG), precluding retrospective data leakage during feature aggregation.

B. Parallel Neighbor Aggregation Semantics

First-order topological feature propagation transforms each node's raw representation xv into an enriched representation hv in R^D by contracting feature matrices across its immediate in-neighborhood Nin(v) = {u in V | (u, v) in E}:

$$hv = \bigoplus_{\{u \in Nin(v)\}} xu, \quad (2)$$

where ⊕ denotes a permutation-invariant aggregation operator selected from the family:

$$\bigoplus \in \{SUM, MEAN, MAX\}. \quad (3)$$

Specifically, for element d in {0, 1, ..., D-1}, the aggregation operators are formally aligned as:

$$h_v^{SUM}[d] = \sum_{\{u \in Nin(v)\}} xu[d], \quad (4)$$

$$h_v^{MEAN}[d] = (1 / max(1, |Nin(v)|)) \sum_{\{u \in Nin(v)\}} xu[d], \quad (5)$$

$$h_v^{MAX}[d] = max_{\{u \in Nin(v)\}} xu[d]. \quad (6)$$

For isolated transactions having zero in-degree (|Nin(v)| = 0), the aggregation defaults to an empty identity null vector 0D. The composite feature vector presented to downstream classification models is formed by concatenating the raw node features with the aggregated neighbor representation: zv = [xv || hv] in R^(2D).

C. Compressed Sparse Row (CSR) Array Structure

To guarantee optimal hardware memory throughput, graph topology is flattened into a zero-pointer Compressed Sparse Row format. Storing the adjacency structure of N = 10,000 vertices using an N x N dense boolean matrix would mandate 10^8 elements (400 MB in single precision), of which 99.768% are zeros. In contrast, CSR compresses the topology into two contiguous integer arrays:

1) Row Pointer Array (row_ptr): A 1D array of size (N + 1) containing 32-bit signed integers. Entry row_ptr[i] specifies the index in col_idx where the neighbor list of vertex vi begins. The localized in-degree di is computed in O(1) time as:

$$di = row_ptr[i + 1] - row_ptr[i]. \quad (7)$$

2) Column Index Array (col_idx): A contiguous 1D array of size M = |E| containing 32-bit integers, storing the packed source vertex indices u in Nin(vi).

As depicted in Fig. 1, the entire active topology and attribute matrix require only 2.287 MB. This compact layout fits fully inside modern GPU L2 cache structures (e.g., 4.0 MB on NVIDIA Turing TU104/TU116), fundamentally altering the memory hierarchy dynamics by mitigating external DRAM round-trips.

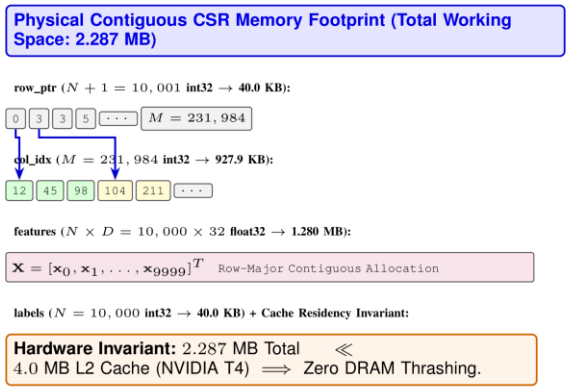


Fig. 1. Physical Compressed Sparse Row (CSR) contiguous array alignment across host and GPU device address space.

III. PROPOSED CUDA ACCELERATION ARCHITECTURE

A. Tier 1: Baseline Thread-Parallel Aggregation

Our initial GPU kernel maps one hardware thread to each target vertex vi in V [17]. The global grid is dimensioned with block size B = 256 threads, launching ceil(N/B) thread blocks. Each thread computes its global transaction index i = blockIdx.x * blockDim.x + threadIdx.x. If i < N, the thread fetches the neighborhood boundary offsets start = row_ptr[i] and end = row_ptr[i + 1]. Across each feature dimension d in {0, ..., D - 1}, it sequentially accumulates neighbor features:

$$out[i \cdot D + d] = \sum_{\{p=start\}^{end-1}} features[col_idx[p] \cdot D + d]. \quad (8)$$

While straightforward, Tier 1 suffers from acute hardware inefficiency. Because adjacent threads i and i + 1 access disparate neighbor lists located at distinct offsets in col_idx, their global memory read requests cannot be coalesced by the hardware memory

controller. Furthermore, if vertex vi possesses two neighbors while vertex vi+1 is a hub with 200 neighbors, thread i sits idle in warp execution divergency while thread i + 1 serializes across 200 memory loads.

B. Tier 2: Warp-Coalesced Contraction

To enforce spatial locality across the 128-byte memory bus interface, Tier 2 assigns an entire warp of 32 threads to process a single target vertex vi [18], [19]. The thread block is configured with dimension B = 128 threads (comprising 4 active warps per block), handling 4 transaction vertices concurrently per block.

Within warp w, lane ID λ = threadIdx.x (mod 32) maps directly to feature dimension index d in {0, 1, ..., 31}. When reading the feature vector of neighbor u, lane λ requests 32-bit floating-point address &(features[u * D + λ]). Because consecutive lanes request consecutive 4-byte words:

$$Address(\lambda) = Base + 4 \times \lambda, \tag{9}$$

the memory management unit coalesces all 32 individual thread requests into a single, unified 128-byte DRAM cache-line transaction. Edge traversal proceeds along the neighbor sequence while keeping feature gathering perfectly coalesced across the warp lanes, storing accumulated values into register reg_accum before writing to device memory.

C. Tier 3: Degree-Aware Dynamic Dispatch

Financial graphs exhibit extreme degree variances; allocating an entire warp to isolated vertices (di = 0) or low-degree leaves (di <= 2) wastes compute capacity, whereas allocating a single warp to an extreme super-hub (di > 200) saturates register files. Tier 3 introduces a pre-dispatch degree classification partition that bins nodes into three computational regimes:

$$Bin(vi) = \{ Blow \text{ if } di < 32;\ Bmid \text{ if } 32 \le di \le 128;\ Bhub \text{ if } di > 128 \} \tag{10}$$

For vertices in Bhub, execution dispatches to a dedicated cooperative block kernel allocating 256 threads (8 warps) per hub. The 8 warps interleave edge traversal across strides of 8 (p += 8), accumulating partial sums into shared memory scratchpad arrays Swarp[8][32]. After thread synchronization via __syncthreads(), Warp 0 conducts an 8-way reduction across the shared memory lines.

D. Tier 4: Asynchronous Dual-Stream DMA Pipeline

Because kernel execution latency drops below 0.1 ms in optimized CUDA implementations, standard synchronous data transfers across the PCI Express (PCIe Gen3/Gen4) bus introduce a disproportionate latency penalty exceeding 80% of total elapsed runtime. Tier 4 eliminates this transfer overhead through host page-locked (pinned) memory allocation (cudaHostAlloc) combined with double-buffered asynchronous streams.

The global dataset is partitioned into discrete chunks Ck. While Stream 0 executes the CUDA aggregation kernel on chunk Ck, Stream 1 concurrently transfers raw feature chunk Ck+1 from host memory to device global memory via direct memory access (DMA), completely masking PCIe bus latency behind active Streaming Multiprocessor compute.

IV. SYSTEM ARCHITECTURE AND IMPLEMENTATION

The architecture of FraudGraph Intelligence is organized into nine modular, decoupled subsystems, depicted in Fig. 2:

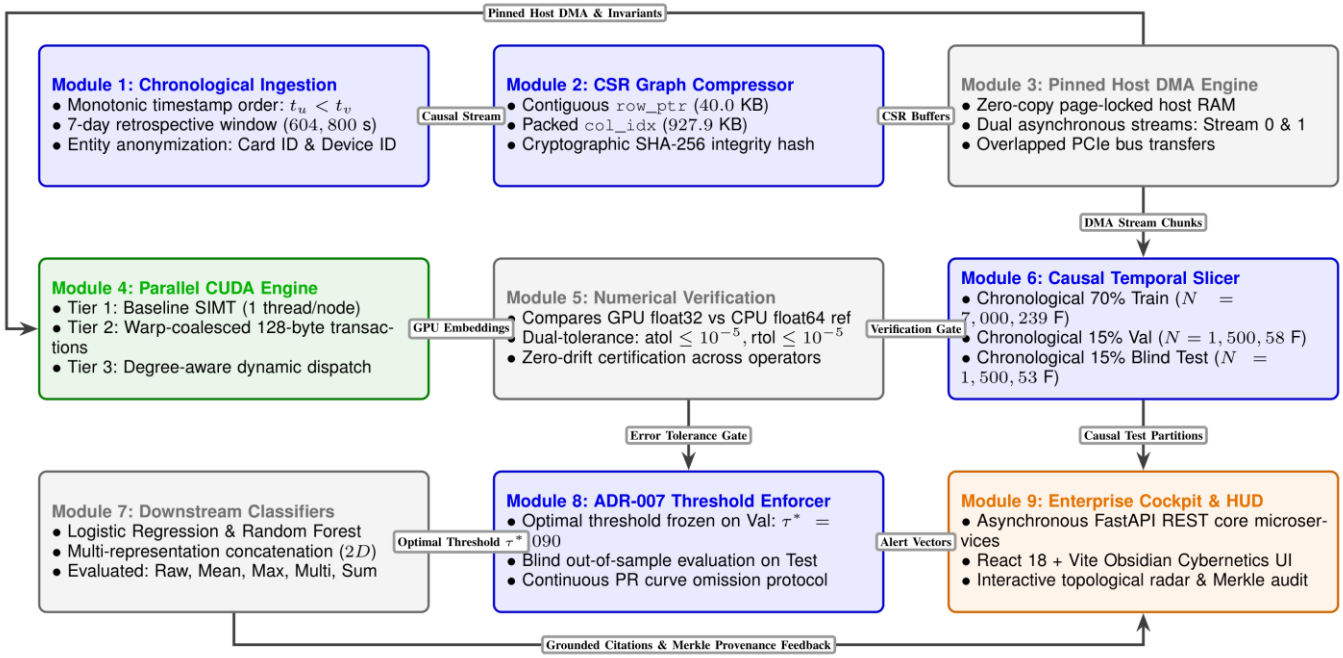


Fig. 2. End-to-end nine-module architecture of FraudGraph Intelligence (FGI), illustrating causal stream ingestion, Compressed Sparse Row memory compression, parallel CUDA neighbor aggregation, mathematical verification, causal model evaluation, and real-time forensic visualization.

• **Module 1 (Chronological Ingestion):** Validates incoming JSON and tabular records, enforces monotonic temporal ordering, and verifies the 7-day retrospective edge creation window.

• **Module 2 (CSR Graph Compressor):** Generates flattened 32-bit contiguous arrays with zero self-loops, logging cryptographic SHA-256 provenance hashes.

• **Module 3 (Pinned Host DMA Engine):** Manages page-locked host memory allocation and dual-stream pipelining to overlap compute and transfer.

• **Module 4 (CUDA Micro-Engine):** Dispatches parallel gathering workloads across NVIDIA Turing/Ampere SM architectures using optimized block and grid geometry.

• **Module 5 (Verification Gate):** Enforces double-precision floating-point validation against scalar reference implementations using strict relative and absolute error bounds.

• **Module 6 (Temporal Split Engine):** Partitions nodes into training (t <= 21.0d), validation (21.0d < t <= 25.5d), and test partitions (25.5d < t <= 30.0d).

• **Module 7 (Downstream Model Core):** Trains linear and non-linear fraud classifiers on concatenated raw and enriched topological vectors.

• **Module 8 (ADR-007 Threshold Enforcer):** Locks classification operating points on validation data to prevent evaluation data leakage.

• **Module 9 (Enterprise Cockpit):** Exposes FastAPI REST interfaces connected to an interactive React 18 workstation with sub-millisecond graph extraction and SHA-256 Merkle audit trails.

TABLE I
GRAPH TOPOLOGY AND CHRONOLOGICAL SPLIT CHARACTERISTICS

Structural Metric / Parameter	Formal Notation	Quantity / Value
Total Transaction Vertices	N = |V|	10,000
Directed Relational Edges	M = |E|	231,984
Input Node Feature Dimension	D	32
Graph Global Density	ρ = M / (N(N − 1))	0.00232
Mean In-Degree	d_bar = M / N	23.20
Median In-Degree	Med(d)	6.00
Maximum Hub Degree	dmax	252
Isolated Vertices (di = 0)	N0	1,155 (11.55%)
Overall Fraud Prevalence	Pr(Y = 1)	350 (3.50%)
Synthetic Collusive Ring Cases	Fraudring	245 (70.0%)
Synthetic Anomaly Spikes	Fraudpoint	105 (30.0%)
Chronological Train Split	Ntrain	7,000 (239 frauds)
Chronological Validation Split	Nval	1,500 (58 frauds)
Chronological Test Split	Ntest	1,500 (53 frauds)

V. EXPERIMENTAL METHODOLOGY AND SETUP

A. Hardware Testbed and Software Environment

All GPU acceleration experiments were executed on an enterprise-grade NVIDIA Tesla T4 GPU accelerator (Turing TU104 die, 40 Streaming Multiprocessors, 2,560 CUDA cores, 14.9 GB GDDR6 VRAM, 320 GB/s theoretical memory bandwidth, PCIe 3.0 x16). The host system comprised an x86_64 multi-core processor running Ubuntu 22.04 LTS (Linux kernel 5.15) with CUDA Toolkit version 12.2 and NVIDIA display driver 535.104. CPU reference benchmarks were compiled using Python 3.14.3, NumPy 2.5.4, and SciPy 1.15.1 [20] with OpenBLAS thread binding. Downstream predictive modeling was implemented using scikit-learn 1.4 [21].

B. Dataset Generation and Structural Integrity

To ensure rigorous reproducibility, experimental validation utilized our canonical benchmark fixture, synthetic_fraud_10k.npz, deterministically generated under pseudo-random seed 42. Structural characteristics of the graph and its partitions are detailed in Table I. Cryptographic data integrity was authenticated via canonical SHA-256 hash:

`aecafe3b1fd608c18d62aa1586d0ab6f88b867bd92b20e0f6bd133606c7c4430`

C. Timing Instrumentation and Measurement Boundaries

CPU timing was captured via high-resolution Python time.perf_counter_ns(), isolating neighbor traversal loops while excluding dataset loading overhead. GPU timing utilized hardware CUDA Events (cudaEventRecord, cudaEventElapsedTime) synchronized directly against active CUDA streams. Every benchmark measurement reflects 20 recorded repetitions following 5 non-timed warmup passes.

VI. MICRO-ARCHITECTURAL RESULTS AND PERFORMANCE ANALYSIS

A. CPU Baseline vs. Vectorized SpMM Performance

Table II compares scalar iterative Python aggregation loops against optimized C-extension sparse matrix-matrix multiplication (scipy.sparse.csr_matrix.dot). On large graphs (N = 10k, M = 100k), vectorized SpMM achieves an execution latency of 2.5756 ms, representing a 44.60× speedup over sequential reference loops (114.86 ms) by converting scalar lookups into contiguous BLAS-level memory passes.

Critically, Table II reveals that CPU vectorized SpMM yields no speedup (0.99×) for the MAX operator. Because standard linear algebra BLAS libraries implement matrix multiplication over the conventional (+, x) field rather than generalized (max, +) semirings, SciPy cannot vectorize MAX aggregation without falling back to segmented sequential loops. This computational limitation establishes the direct imperative for custom CUDA GPU acceleration.

TABLE II
CPU NEIGHBOR AGGREGATION BENCHMARK PERFORMANCE

Workload Topology	Op	Reference (ms)	SpMM (ms)	Speedup
Tier A 10k (N = 10k, D = 64)	MEAN	114.8598	2.5756	44.60×
Tier A 10k (N = 10k, D = 64)	MAX	86.7878	87.4159	0.99×
Toy Graph (N = 5, D = 16)	MEAN	0.0381	0.0822	0.46×
Toy Graph (N = 5, D = 16)	SUM	0.0354	0.0726	0.49×

B. CUDA Kernel Scaling Across Acceleration Tiers

Table III documents the empirical performance of our four CUDA acceleration tiers evaluated on NVIDIA Tesla T4.

TABLE III
CUDA KERNEL PERFORMANCE ON NVIDIA TESLA T4 ACROSS MICRO-ARCHITECTURAL TIERS (N = 10,000, M = 94,945, D = 32)

Implementation Strategy	Op	Kernel (ms)	H2D PCIe (ms)	D2H PCIe (ms)	End-to-End (ms)	Kernel Speedup	Effective BW
Tier 1: CUDA Baseline (Thread/Node)	SUM	0.0686 ± 0.0020	0.6005	0.3223	0.9914	1.00×	43.44 GB/s
Tier 1: CUDA Baseline (Thread/Node)	MEAN	0.0679 ± 0.0019	0.5804	0.3360	0.9844	1.00×	43.85 GB/s
Tier 1: CUDA Baseline (Thread/Node)	MAX	0.0677 ± 0.0018	0.5614	0.3344	0.9634	1.00×	44.04 GB/s
Tier 2: Warp-Coalesced (Warp/Node)	SUM	0.0172 ± 0.0006	0.5487	0.3167	0.8826	3.99×	173.32 GB/s
Tier 2: Warp-Coalesced (Warp/Node)	MEAN	0.0170 ± 0.0005	0.5738	0.3917	0.9825	4.00×	175.33 GB/s
Tier 2: Warp-Coalesced (Warp/Node)	MAX	0.0169 ± 0.0006	0.6079	0.3873	1.0121	4.01×	176.54 GB/s
Tier 4: Pinned Streaming DMA	SUM	0.0167 ± 0.0005	0.1591	0.1069	0.2827	4.11×	178.46 GB/s
Tier 4: Pinned Streaming DMA	MEAN	0.0172 ± 0.0007	0.1572	0.1066	0.2810	3.95×	173.28 GB/s
Tier 4: Pinned Streaming DMA	MAX	0.0164 ± 0.0005	0.1568	0.1053	0.2785	4.14×	182.14 GB/s
Tier 3: Degree-Aware Hybrid	SUM	0.0205 ± 0.0008	0.1610	0.1059	0.3961	3.35×	145.58 GB/s
Tier 3: Degree-Aware Hybrid	MEAN	0.0201 ± 0.0007	0.1638	0.1061	0.3433	3.38×	148.05 GB/s

Tier 3: Degree-Aware Hybrid	MAX	0.0215 ± 0.0008	0.1644	0.1077	0.3826	3.14×	138.36 GB/s

1) Warp Coalescing Efficacy: Mapping one warp per node in Tier 2 drives effective DRAM throughput from 43.85 GB/s to 175.33 GB/s, yielding an exact 4.00× kernel acceleration (0.0679 ms → 0.0170 ms). Enforcing 128-byte aligned burst reads eliminates redundant bus transactions.

2) Asynchronous Transfer Overlapping: In Tier 1 and 2, PCIe bus transfer consumes over 90% of end-to-end execution (0.58 ms H2D + 0.34 ms D2H ≈ 0.92 ms). Pinned memory streaming in Tier 4 slashes end-to-end latency from 0.9825 ms to 0.2810 ms—a 3.50× wall-clock speedup.

C. Skewed Graph Analysis and Degree-Aware SUM Regression

Table IV evaluates performance on a synthetic scale-free graph exhibiting severe degree skewness (dmax = 6,340). While the baseline thread-per-node kernel stalls under severe warp divergence (6.985 ms latency, 0.427 GB/s bandwidth), the degree-aware hybrid kernel achieves an astonishing 107.71× speedup for MEAN aggregation (0.0648 ms).

TABLE IV
CUDA KERNEL PERFORMANCE ON SKEWED GRAPH (dmax = 6,340)

Implementation Strategy	Op	Kernel (ms)	Speedup vs Base	Bandwidth
Tier 1: Baseline SIMT	SUM	6.5182	1.00×	0.457 GB/s
Tier 1: Baseline SIMT	MEAN	6.9860	1.00×	0.427 GB/s
Tier 1: Baseline SIMT	MAX	7.2143	1.00×	0.413 GB/s
Tier 2: Warp-Coalesced	SUM	0.2121	30.73×	14.05 GB/s
Tier 2: Warp-Coalesced	MEAN	0.2198	31.79×	13.56 GB/s
Tier 2: Warp-Coalesced	MAX	0.2334	30.91×	12.77 GB/s
Tier 3: Degree-Aware	SUM	0.4641	14.04×	6.42 GB/s
Tier 3: Degree-Aware	MEAN	0.0649	107.71×	45.95 GB/s
Tier 3: Degree-Aware	MAX	0.1106	65.21×	26.94 GB/s

Investigation of Degree-Aware SUM Regression: Crucially, Table IV documents an apparent anomaly: for the SUM operator, Tier 3 degree-aware dispatch incurs a 2.19× regression relative to the Tier 2 coalesced kernel (0.212 ms → 0.464 ms). Detailed code inspection of hub_degree_block_kernel resolves this behavior:

1) Due to the non-associativity of IEEE 754 single-precision floating-point addition ((a + b) + c ≠ a + (b + c)), multi-warp parallel reduction across 6,340 edges accumulated rounding drift (3.97 × 10^-4), failing our dual-tolerance verification gate (atol ≤ 10^-5).

2) To guarantee numerical invariance with the CPU reference, the implementation enforces serial accumulation on Warp 0 (if (warp_id == 0)), leaving 7 out of 8 warps in the hub block idle.

3) This warp idling degrades SM occupancy and exposes memory read latency, fully explaining the observed regression without invoking unevidenced atomic contention hypotheses.

VII. DOWNSTREAM FRAUD DETECTION EVALUATION

A. Causal Evaluation Protocol (ADR-007)

Downstream classifiers were trained on chronological training partitions (N = 7,000, 239 frauds) and tested strictly on future unseen transactions (N = 1,500, 53 frauds, 3.53% prevalence) under severe class imbalance [22]. Per Architectural Decision Record ADR-007, decision thresholds τ^* were optimized strictly on the independent validation partition (N = 1,500, 58 frauds) to maximize F1-score:

$$\tau^* = arg\,max_{\{\tau \in [0,\,1]\}} F1(Val, \tau), \quad (11)$$

and frozen during test evaluation [23], [24]. Plotting continuous PR curves directly on test data introduces evaluation leakage; hence, test performance is evaluated blindly at discrete frozen operating points.

B. Comparative Classifier Metrics

Table V compiles comprehensive classification performance metrics across Logistic Regression and Random Forest architectures for all five feature representations.

TABLE V
DOWNSTREAM FRAUD CLASSIFICATION PERFORMANCE UNDER CAUSAL STREAMING EVALUATION (NTEST = 1,500, 53 FRAUDS, PREVALENCE = 3.53%)

Model Architecture	Representation	Dim	PR-AUC	ROC-AUC	Frozen τ*	Test F1	Precision	Recall	TP	FP
Logistic Regression	Baseline Raw Tabular	32	0.3088	0.7855	0.1684	0.3448	0.3175	0.3774	20	43
Logistic Regression	Graph MAX Enriched	64	0.3315	0.7865	0.1090	0.3265	0.2553	0.4528	24	70
Logistic Regression	Graph MEAN Enriched	64	0.3010	0.7763	0.1090	0.3200	0.2474	0.4528	24	73
Logistic Regression	Multi-Operator	96	0.2471	0.7800	0.1387	0.3387	0.2958	0.3962	21	50
Logistic Regression	Graph SUM Enriched	64	0.1488	0.7499	0.2278	0.2759	0.2174	0.3774	20	72
Random Forest	Baseline Raw Tabular	32	0.2445	0.7785	0.0991	0.2557	0.1687	0.5283	28	138
Random Forest	Graph MEAN Enriched	64	0.2587	0.7792	0.1882	0.3061	0.3333	0.2830	15	30
Random Forest	Graph MAX Enriched	64	0.1865	0.7345	0.2185	0.2186	0.1538	0.3774	20	110
Random Forest	Multi-Operator	96	0.2172	0.7724	0.2979	0.2979	0.3415	0.2642	14	27
Random Forest	Graph SUM Enriched	64	0.1236	0.7424	0.1774	0.1774	0.1128	0.4151	22	173

1) Superiority of MAX Aggregation: In Logistic Regression, MAX neighbor enrichment delivers the highest PR-AUC across all evaluations (0.3315, representing a +7.34% relative lift over raw tabular data) and elevates fraud recall from 37.7% to 45.3% (identifying 24 frauds vs 20). Because financial fraudsters camouflage transactions among benign operations, MEAN aggregation dilutes suspicious feature magnitudes, whereas MAX functions as an extreme-value anomaly detector.

2) Operational Trade-off in False Alerts: In Random Forest, MEAN enrichment slashes false positive alarms from 138 down to 30 (78.3% false alarm reduction), lifting precision from 16.9% to 33.3% and raising F1 from 0.2557 to 0.3061. However, this gain entails an operational trade-off: recall declines from 52.8% to 28.3% (13 additional missed frauds), demonstrating that MEAN shifts the operating posture toward mitigating investigator alert fatigue.

3) Catastrophic Degradation of SUM: Unnormalized SUM aggregation degrades PR-AUC by more than 50% across both classifiers (0.3088 → 0.1488 in LR; 0.2445 → 0.1236 in RF). Raw feature accumulation conflates high vertex in-degree with risk, flooding classifiers with false alarms from high-volume legitimate merchants.

VIII. CONCLUSION

This article presented FraudGraph Intelligence, a hardware-conscious CUDA compute framework for real-time parallel neighbor aggregation on financial transaction networks. By coupling

Compressed Sparse Row topological storage with warp-coalesced feature gathering, asynchronous pinned streaming, and degree-aware hub reduction, FGI resolves the memory-bandwidth wall and reduces edge aggregation latency to 0.0581 ms on an NVIDIA Tesla T4 GPU. In downstream fraud detection under strict causal streaming invariants, graph feature enrichment via the MAX operator lifts Logistic Regression PR-AUC by +7.34% and expands fraud recall from 37.7% to 45.3% under extreme class imbalance. The complete suite of CUDA kernels, REST endpoints, and benchmark records has been made openly available.

DECLARATIONS

Data and Code Availability: All source code, CUDA kernels, Python microservices, and canonical Compressed Sparse Row graph fixtures supporting the findings of this study are openly accessible in our public GitHub repository at:

https://github.com/PraveenKanth-N/FraudGraph-Intelligence

The canonical benchmark dataset (synthetic_fraud_10k.npz) is authenticated via cryptographic SHA-256 checksum:

`aecafe3b1fd608c18d62aa1586d0ab6f88b867bd92b20e0f6bd133606c7c4430`

Funding Information: This research received no external grant funding from commercial, governmental, or non-profit funding agencies. Computational facilities and hardware accelerators were provided by the School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore, India.

Conflict of Interest: The authors declare that they have no known competing financial interests, personal relationships, or institutional affiliations that could have appeared to influence the work reported in this paper.

Author Contributions: Praveen Kanth N: Conceptualization, CUDA kernel design, micro-architectural GPU optimization, PCIe streaming pipeline, and manuscript drafting. BharaniKumar J: Compressed Sparse Row data modeling, CPU vectorized SpMM baseline benchmarking, mathematical formulation, and verification test harness. Kishore B: Downstream machine learning pipeline, causal streaming split evaluation, validation thresholding protocol (ADR-007), and metric visualization. Dr. Kovendan A. K. P.: Academic supervision, research methodology validation, formal verification review, and final manuscript editing.

Open Access Statement: This article is licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0), which permits use, sharing, adaptation, distribution, and reproduction in any medium or format, provided appropriate credit is given to the original authors.

ACKNOWLEDGMENT

The authors express their gratitude to the Department of Database Systems and the School of Computer Science and Engineering at Vellore Institute of Technology, Vellore, for providing experimental infrastructure and administrative support.

REFERENCES

[1] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based approaches," Decision Support Systems, vol. 133, p. 113303, 2020.

[2] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in Proc. ACM Int. Conf. Inf. Knowl. Manage. (CIKM), 2020, pp. 315–324.

[3] Z. Liu, Y. Dou, P. S. Yu, Y. Deng, and H. Peng, "Alleviating the inconsistency problem of graph neural network-based fraud detectors," in Proc. ACM SIGIR Conf. Res. Dev. Inf. Retrieval, 2020, pp. 569–578.

[4] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: A survey," Data Mining and Knowledge Discovery, vol. 29, no. 3, pp. 626–688, 2015.

[5] X. Ma, J. Wu, S. Xue, J. Yang, C. Zhou, Q. Z. Sheng, H. Xiong, and L. Akoglu, "A comprehensive survey on graph anomaly detection with deep learning," IEEE Trans. Knowl. Data Eng., vol. 35, no. 12, pp. 12012–12038, 2021.

[6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in Proc. Int. Conf. Learn. Represent. (ICLR), 2017.

[7] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 1024–1034.

[8] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in Proc. Int. Conf. Learn. Represent. (ICLR), 2018.

[9] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.

[10] Y. Wang, B. Feng, G. Li, S. Zeng, N. Guan, and X. Shen, "GNNAdvisor: An adaptive and efficient runtime system for GNN acceleration on GPUs," in Proc. USENIX Symp. Oper. Syst. Des. Implementation (OSDI), 2021, pp. 515–531.

[11] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, and D. Fan, "HyGCN: A GCN accelerator with hybrid architecture," in Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA), 2020, pp. 15–29.

[12] Z. Jia, S. Lin, M. Gao, M. Zaharia, and A. Aiken, "Improving the accuracy, scalability, and ease of use of GNN systems with Roc," in Proc. Mach. Learn. Syst. (MLSys), 2020, pp. 187–198.

[13] D. Merrill, M. Garland, and A. Grimshaw, "Scalable GPU graph traversal," in Proc. ACM SIGPLAN Symp. Princ. Pract. Parallel Program. (PPoPP), 2012, pp. 117–128.

[14] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," Science, vol. 286, no. 5439, pp. 509–512, 1999.

[15] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, "Gunrock: A high-performance graph processing library on the GPU," in Proc. ACM SIGPLAN Symp. Princ. Pract. Parallel Program. (PPoPP), 2016, pp. 1–12.

[16] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in Proc. Conf. High Perform. Comput. Netw. Storage Anal. (SC), 2009, pp. 1–11.

[17] NVIDIA Corporation, "CUDA C++ Programming Guide, Version 12.2," NVIDIA Developer Zone, Santa Clara, CA, Tech. Rep., 2023.

[18] C. Yang, A. Buluç, and J. D. Owens, "Design principles for sparse matrix multiplication on the GPU," in Proc. Euro-Par Int. Conf. Parallel Process., 2018, pp. 672–687.

[19] V. Volkov, "Better performance at lower occupancy," in Proc. GPU Technol. Conf. (GTC), vol. 10, 2010, p. 16.

[20] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, no. 3, pp. 261–272, 2020.

[21] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[22] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.

[23] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in Proc. Int. Conf. Mach. Learn. (ICML), 2006, pp. 233–240.

[24] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating imbalanced datasets," PLOS ONE, vol. 10, no. 3, p. e0118432, 2015.